Zellic

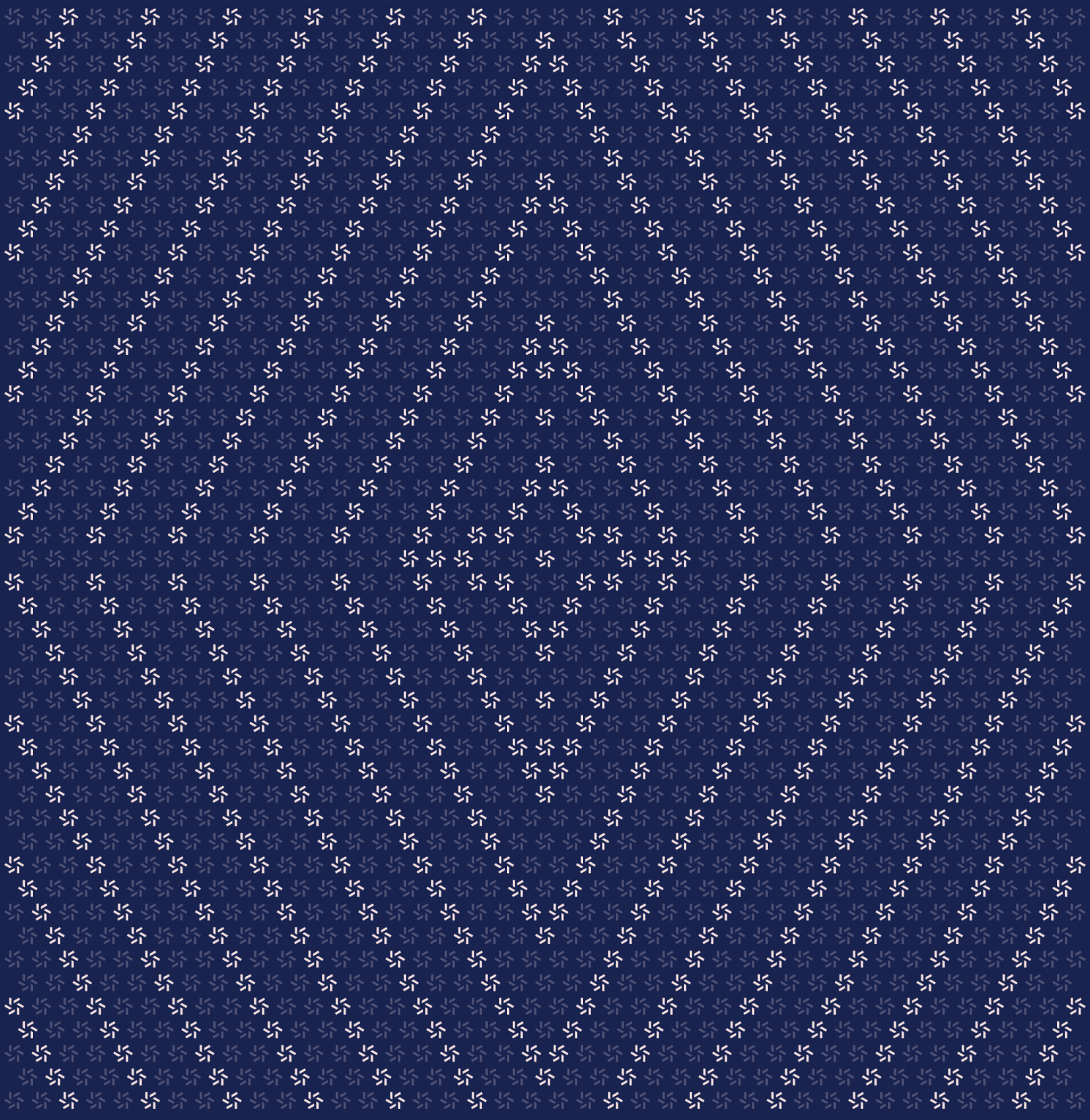
Prepared for
Usmann Khan
Hans Behrens
Plasma

Prepared by
Huw Grano
Chongyu Lv
Zellic

July 8, 2026

Plasma

Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Plasma from July 1st to July 2nd, 2026. During this engagement, Zellic reviewed Plasma's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does ValidatorSet use OpenZeppelin's upgradability and ownership libraries correctly for its intended proxy deployment model?
- Can ValidatorSet become stuck in a state where ownership, upgradability, or validator-set updates are no longer operational?
- Are any functions that modify the validator set or implementation exposed to unintended callers rather than the intended owner/governance role?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Plasma contracts, we discovered one finding, which was of low impact.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	0

2. Introduction

2.1. About Plasma

Plasma contributed the following description of Plasma:

> Plasma is a high-performance, EVM-compatible Layer 1 blockchain designed to optimize stablecoin settlement. Utilizing a customized implementation of the Fast Hotstuff consensus protocol, Plasma ensures low-latency transactions and deterministic finality, facilitating seamless integration with token issuers and financial applications.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Plasma Contracts

Type	Solidity
Platform	EVM-compatible
Target	plasma-consensus
Repository	https://github.com/PlasmaLaboratories/plasma-consensus ↗
Version	`06b061403bcc7e6f050ddae58e1f61273c3e1f55`
Programs	`Placeholder.sol` `ValidatorSet.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 0.5 person-weeks. The assessment was conducted by two consultants over the course of two calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io

The following consultants were engaged to conduct the assessment:

Huw Grano
Engineer
huw@zellic.io

Chongyu Lv
Engineer
chongyu@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 1, 2026	Start of primary review period
July 2, 2026	End of primary review period

3. Detailed Findings

3.1. Gas-heavy validator-set updates may be difficult to include during congestion

Target	ValidatorSet		
Category	Optimization	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `ValidatorSet.setValidators` function replaces the full validator set in one transaction. Before writing the new set, it validates each BLS public key length and performs an `O(n^2)` duplicate check. It then deletes the existing storage array and pushes each validator into storage.

```
function setValidators(Validator[] calldata newValidators)
    external onlyOwner {
    uint256 n = newValidators.length;
    if (n < MIN_VALIDATORS) {
        revert TooFewValidators(MIN_VALIDATORS, n);
    }
    if (n > MAX_VALIDATORS) {
        revert TooManyValidators(MAX_VALIDATORS, n);
    }
    _validateValidators(newValidators);

    ValidatorSetStorage storage $ = _getValidatorSetStorage();
    delete $.validators;
    for (uint256 i = 0; i < n; i++) {
        $.validators.push(newValidators[i]);
    }

    emit ValidatorsUpdated(n);
}

function _validateValidators(Validator[] calldata newValidators)
    private pure {
    bytes32[] memory seenPubkeyHashes = new bytes32[](newValidators.length);

    for (uint256 i = 0; i < newValidators.length; i++) {
        bytes calldata pubkey = newValidators[i].blsPubkey;
        if (pubkey.length != BLS_PUBKEY_LENGTH) {
            revert InvalidPubkeyLength(BLS_PUBKEY_LENGTH, pubkey.length);
        }
```

```
            bytes32 pubkeyHash = keccak256(pubkey);
            for (uint256 j = 0; j < i; j++) {
                if (seenPubkeyHashes[j] == pubkeyHash) {
                    revert DuplicatePubkey(i);
                }
            }
            seenPubkeyHashes[i] = pubkeyHash;
        }
    }
```

Local gas measurements at the maximum supported validator count show

- an empty set to 256 validators: 21,327,163 gas
- 256-validator set to a new 256-validator set: 19,954,302 gas
- `getValidators()` with 256 validators: 564,514 gas

The update transaction remains below Plasma's 36,000,000 block gas limit, so this is not a direct block-gas-limit failure. The concern is operational: a max-size validator-set update consumes more than half of a block and may be harder for block proposers or transaction-selection policy to include during heavy congestion unless the protocol or operators have a priority path for this governance transaction.

Impact

If governance needs to update the validator set shortly before a snapshot or epoch boundary, a high-gas `setValidators` transaction may be delayed by normal block-construction incentives or network congestion. In that case, the intended validator-set update can miss the target snapshot and be deferred to a later epoch.

This does not let an unprivileged user modify the validator set, does not directly cause asset loss, and does not make `getValidators()` exceed the consensus call gas limit. The impact is limited to delayed validator-set activation and related operational liveness risk under congestion.

In summary, congestion or block-construction policy can delay a max-size owner/governance `setValidators` transaction, causing a validator-set update to miss its intended activation window when no priority inclusion path exists.

Recommendations

Consider reducing the worst-case gas cost of `setValidators` if the supported validator-set size is expected to grow beyond the current operational target. For example, the contract could require validator public keys to be provided in sorted order and reject duplicates by comparing only adjacent entries, replacing the current quadratic duplicate scan with a linear check.

If Plasma intends to operate with a much smaller validator set in the near term, consider lowering `MAX_VALIDATORS` to match that operational assumption or documenting the operational

requirement that high-gas validator-set updates have a reliable inclusion path before the relevant snapshot or epoch boundary.

Remediation

This issue has been acknowledged by Plasma.

4. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

4.1. Module: ValidatorSet.sol

Function: `_validateValidators(Validator[] newValidators)`

This function validates that every entry in `newValidators` has a 48-byte BLS pubkey and that no two entries share the same pubkey. The function does not check that the pubkeys represent valid BLS public keys other than the length requirement.

Inputs

- `newValidators`
  - **Control**: Full control by the owner through `setValidators`.
  - **Constraints**: The caller enforces the validator-count bounds before calling this function. This function enforces the pubkey-length and uniqueness constraints described below.
  - **Impact**: Determines whether `setValidators` proceeds to update storage or reverts.

Branches and code coverage

Intended branches

- ☑ Every `newValidators[i].blsPubkey` should be exactly 48 bytes, and no two entries should share the same pubkey hash.

Negative behavior

- ☑ Revert with `InvalidPubkeyLength` if any `blsPubkey` is shorter than 48 bytes.
- ☑ Revert with `InvalidPubkeyLength` if any `blsPubkey` is longer than 48 bytes.
- ☑ Revert with `DuplicatePubkey` if any two entries share the same `blsPubkey` hash.

Function: `constructor()`

This function disables initializers on the implementation contract itself so that it can never be initialized directly.

Branches and code coverage

Intended branches

- ☐ The implementation constructor should call `_disableInitializers()`, preventing future direct initialization or reinitialization of the implementation contract.

Negative behavior

- None. The constructor does not revert.

Function: `getValidators()`

This function returns the full array of currently active validators exactly as stored, in insertion order. As described in the interface documentation, callers (the consensus clients) are responsible for sorting the returned set by pubkey.

Branches and code coverage

Intended branches

- ☑ The function should return an empty array before the first validator-set update.
- ☑ The function should return validators in the same insertion order stored by `setValidators`.
- ☑ The raw `0xb7ab4db5` selector should return ABI-decodable `Validator[]` data.
- ☑ The function should stay below the consensus call gas limit at the maximum supported validator count.

Negative behavior

- None. The function performs no validation and does not revert.

Function: `initialize(address initialOwner)`

This is an one-time use function that sets the contract's owner via the OpenZeppelin Ownable/Ownable2Step initialization chain.

Inputs

- `initialOwner`
  - **Control**: Full control by the deployer during initialization.
  - **Constraints**: Must be nonzero — enforced by the OpenZeppelin library.
  - **Impact**: Becomes the contract's owner, the only account permitted to call `setValidators`, transfer ownership, and authorize UUPS upgrades through

the inherited `upgradeToAndCall` flow.

Branches and code coverage

Intended branches

- ☑ A proxy deployment with a nonzero `initialOwner` should set that address as the owner.
- ☑ The inherited `owner()` view should return the initialized owner.
- ☑ The current owner should be able to start a two-step ownership transfer through `transferOwnership`.
- ☑ The pending owner should not gain access to owner-only `setValidators` before calling `acceptOwnership`.
- ☑ The pending owner should become the owner after `acceptOwnership`, and the previous owner should lose access to the owner-only `setValidators`.
- ☑ The current owner should be able to upgrade the proxy through the inherited UUPS upgrade flow while preserving validator-set state.

Negative behavior

- ☑ Revert if `initialize` is called more than once.
- ☑ Revert if a non-owner attempts to upgrade the proxy through the inherited UUPS upgrade flow.
- ☐ Revert if `initialOwner == address(0)`.

Function: `renounceOwnership()`

This function overrides Ownable's `renounceOwnership` to unconditionally revert, so ownership of the validator-set registry can never be renounced and the contract can never be left without an owner/upgrade authority.

Branches and code coverage

Intended branches

- None. This function has no success path as its only behavior is an unconditional revert (see below).

Negative behavior

- ☑ Revert with `RenounceOwnershipDisabled` when the owner attempts to renounce ownership.
- ☐ Revert with `RenounceOwnershipDisabled` when a non-owner attempts to renounce ownership.

Function: `setValidators(Validator[] newValidators)`

This function replaces the entire on-chain validator set with `newValidators` (after validations are performed) by repopulating array storage entry by entry.

Inputs

- `newValidators`
  - **Control**: Full control by the owner.
  - **Constraints**: Length must be within `[MIN_VALIDATORS, MAX_VALIDATORS]` (4–256 inclusive); each `blsPubkey` must be exactly 48 bytes; no two entries may share the same `blsPubkey`.
  - **Impact**: Becomes the new complete validator set returned by `getValidators()` — consensus clients derive the next-epoch committee from it as described in the Aquila spec.

Branches and code coverage

Intended branches

- ☑ The owner should be able to set a valid validator set.
- ☑ The owner should be able to set the minimum supported validator count.
- ☑ The owner should be able to set the maximum supported validator count.
- ☑ A valid update should replace the prior validator set and clear stale storage entries.
- ☑ The function should preserve the provided insertion order in storage.
- ☑ The function should emit `ValidatorsUpdated` with the new validator count.
- ☑ The maximum-size update should stay below the configured update gas limit.

Negative behavior

- ☑ Revert with `OwnableUnauthorizedAccount` if the caller is not the contract owner.
- ☑ Revert with `TooFewValidators` if `newValidators.length < MIN_VALIDATORS`.
- ☑ Revert with `TooManyValidators` if `newValidators.length > MAX_VALIDATORS`.
- ☑ Revert with `InvalidPubkeyLength` if any `blsPubkey` is shorter than 48 bytes.
- ☑ Revert with `InvalidPubkeyLength` if any `blsPubkey` is longer than 48 bytes.
- ☑ Revert with `DuplicatePubkey` if any two entries share the same `blsPubkey`.
- ☑ Preserve the prior validator set if a validator update reverts.

Function: `validatorCount()`

This function returns the number of validators currently stored in the registry.

Branches and code coverage

Intended branches

- ☑ The function should return zero before the first validator-set update.
- ☑ The function should return the current validator count after a successful validator-set update.

Negative behavior

- None. The function does not revert.

Function: `version()`

This function returns the contract's version identifier as a left-aligned ASCII string packed into `bytes32`.

Branches and code coverage

Intended branches

- ☑ The function should return `bytes32("plasma-validator-set/v1")`.

Negative behavior

- No negative behavior is possible; the function requires that the hardcoded string literal does not exceed 32 bytes, but this revert is unreachable unless the source code itself is edited.

5. Assessment Results

During our assessment on the scoped Plasma contracts, we discovered one finding, which was of low impact.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.